MoonPark Pricing Engine

A secure, extensible parking-pricing API

Complex rules.
Clear answers.

Java 25 / Spring Boot / PostgreSQL / Redis

CORRECT | SECURE | EXTENSIBLE



Same parking session. Different pricing logic.

The zone decides how time becomes money.

M1 / MINUTES



1 started minute = 1 NOK

61 minutes > 61 NOK

M2 / HOURS



Weekday 100 / Weekend 200

NOK per started hour

M3 / TIME WINDOWS

MON-SAT / 08:00-16:00



First 60 daytime min free

Then 2 NOK/started min

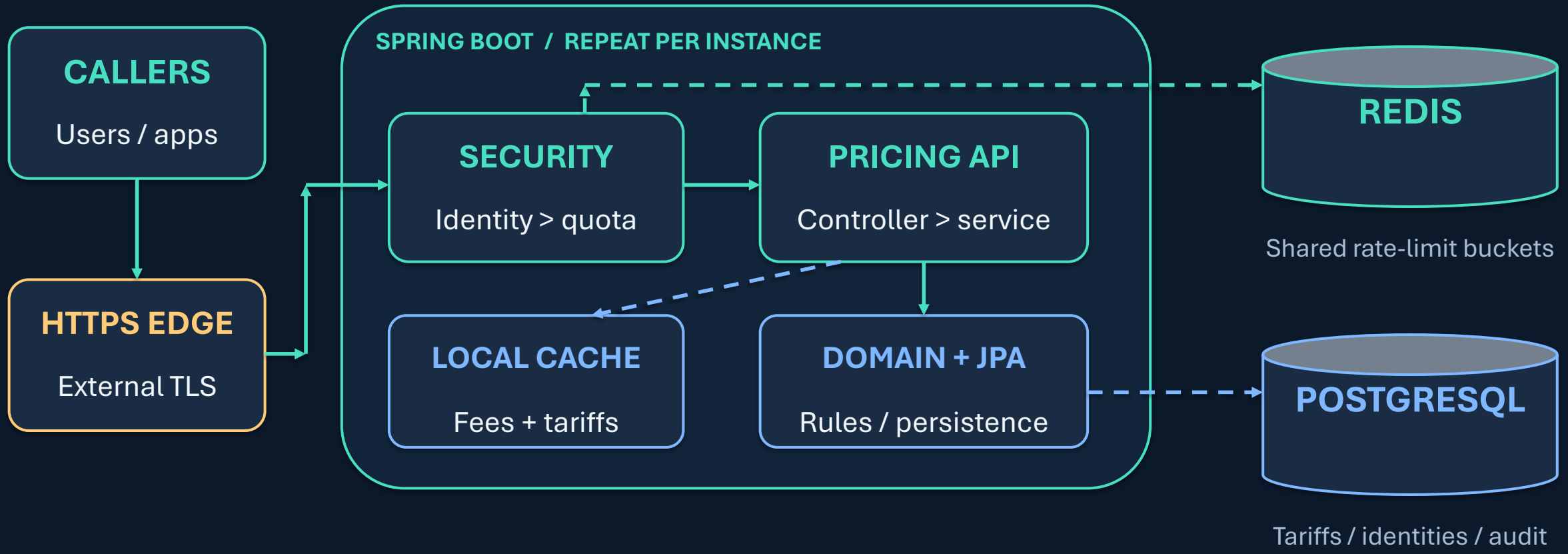
Outside: 3 NOK/min | Sun: free

Not just elapsed time. Started units, calendar boundaries and tariff versions.

Sources: README.md; model/domain/tariff | Seeded default rules

One application. Clear responsibilities.

Solid lines: request path. Dashed lines: supporting data and shared state.

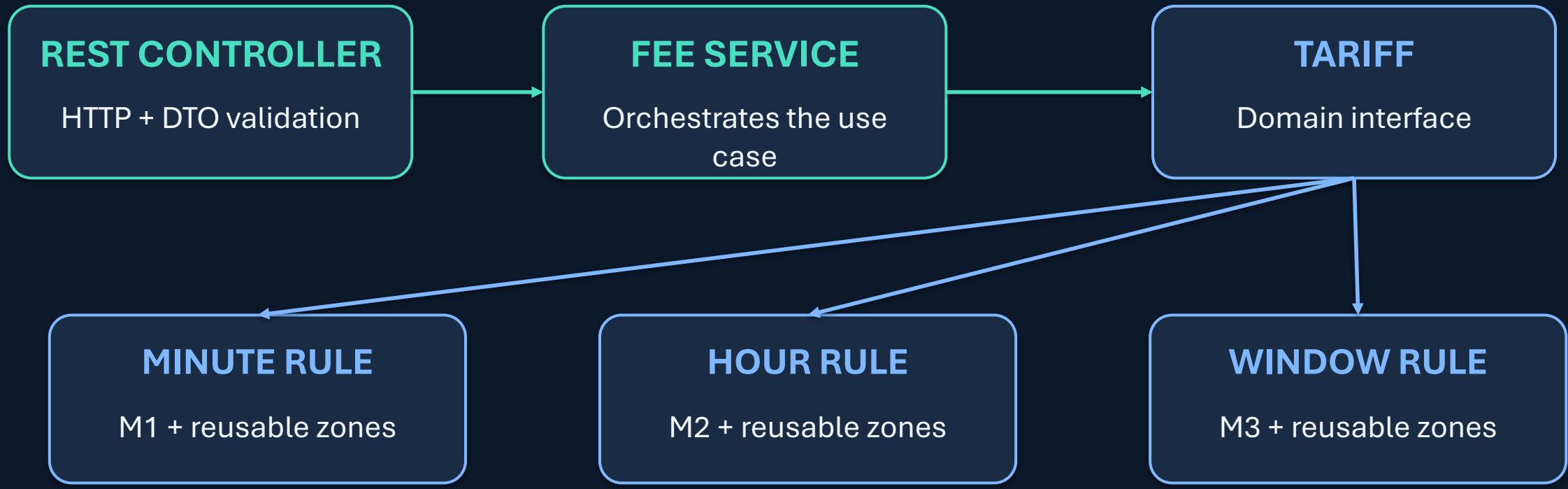


Fast local pricing. Shared identity data. Distributed request limits.

Sources: SecurityConfig; FeeCalculationService; CachedTariffRegistry; application.yml

Add zones without rewriting the engine.

Pricing behavior is isolated behind the Tariff interface.

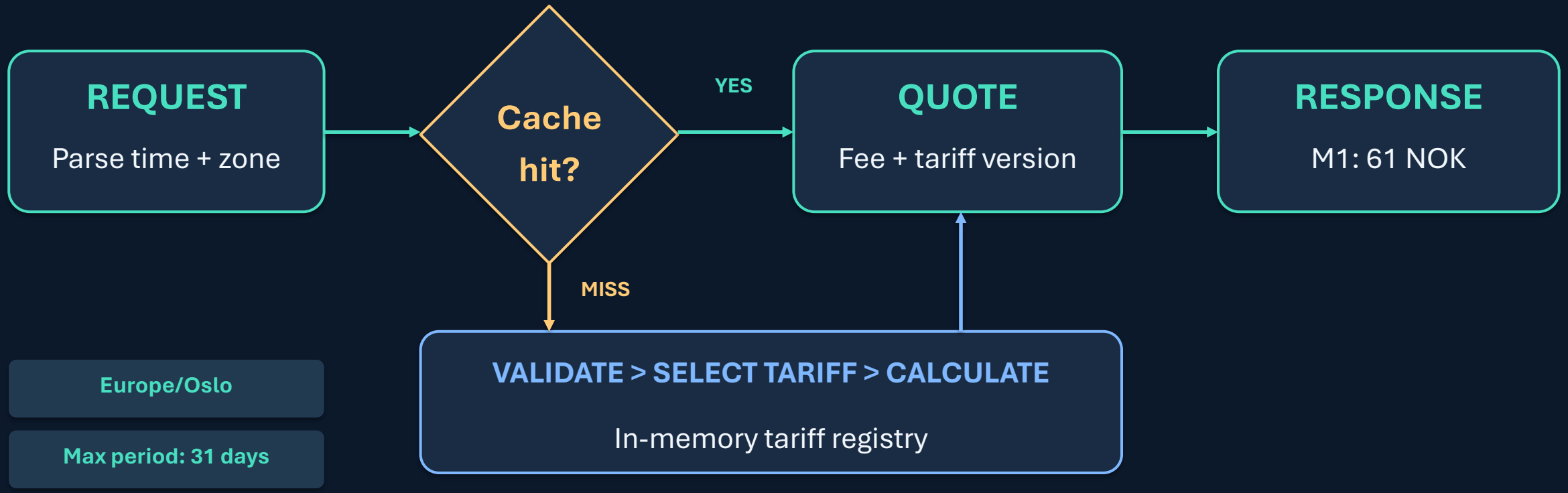


New zone: configure a rule. New algorithm: add a strategy.

Sources: CalculateFeeUseCase; FeeCalculationService; TariffFactory; Tariff implementations

The fastest database call is the one you avoid.

GET /api/v1/takst | zone + start + end

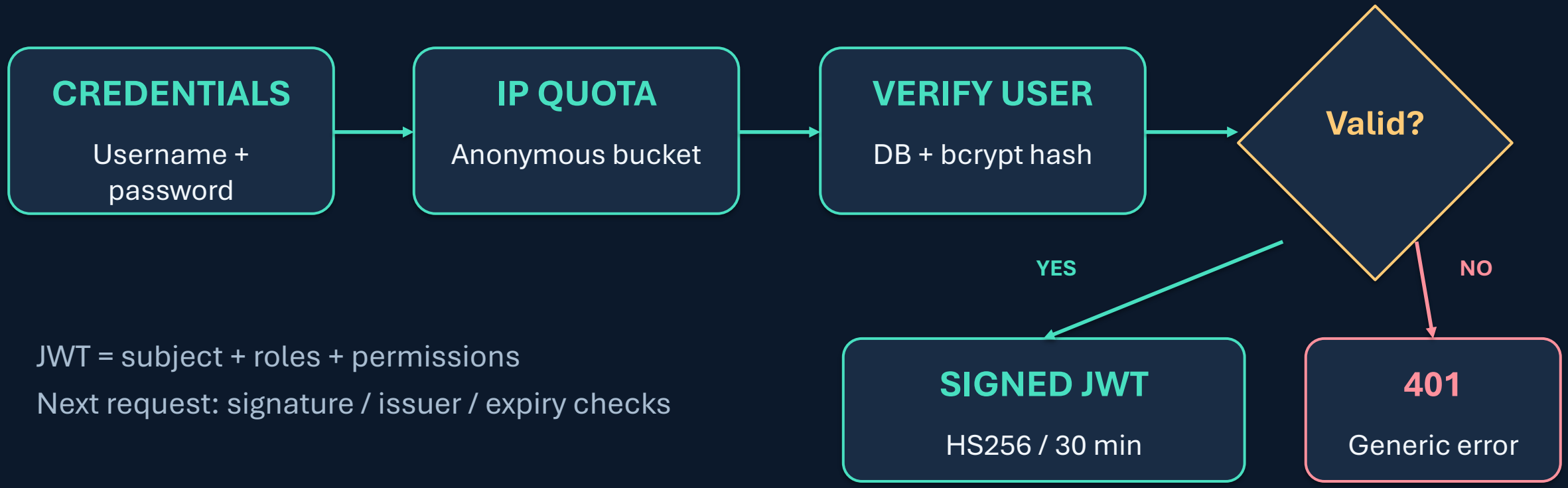


The session START selects the tariff version.

Sources: TariffController; FeeCalculationService; CachedTariffRegistry; application.yml

Authenticate once. Carry identity in a token.

POST /api/v1/auth/login



JWT = subject + roles + permissions

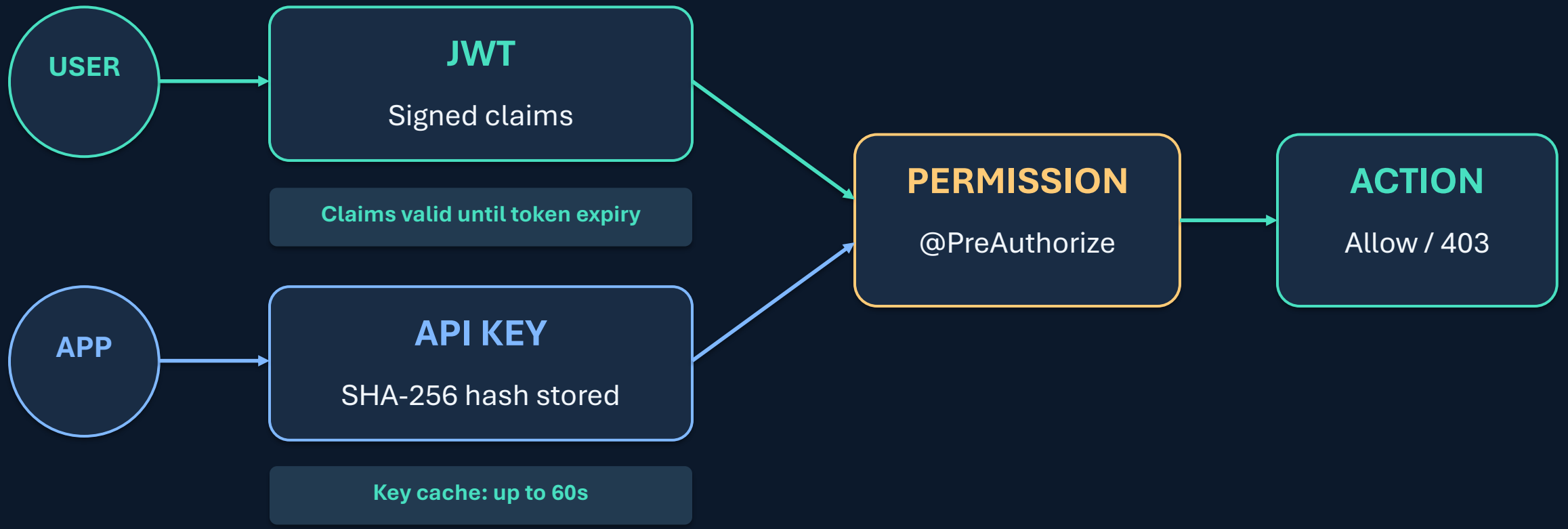
Next request: signature / issuer / expiry checks

Unknown user? A dummy hash check reduces enumeration signals.

Sources: AuthService; TokenService; JwtConfig; RateLimitFilter

Two identities. One permission gate.

Authentication establishes who. Authorization decides what.

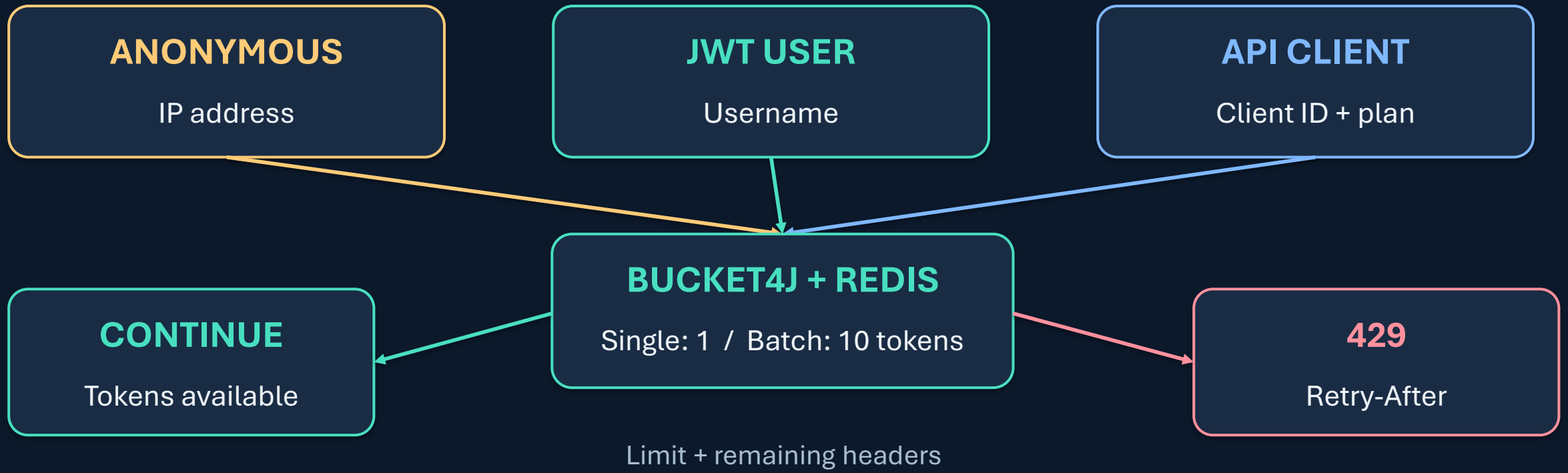


Choose JWT OR API key. Never both.

Sources: ApiKeys; ApiClientService; SecurityConfig; controller @PreAuthorize

Every caller has a budget.

Authentication > identify caller > consume tokens > authorize

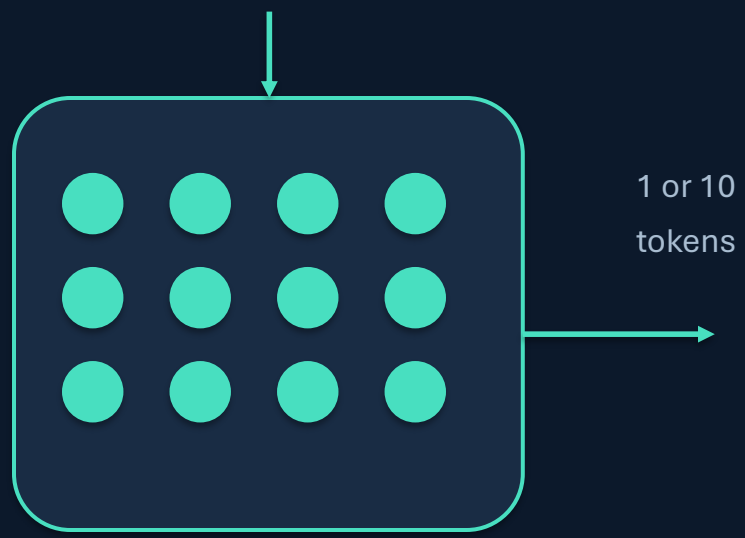


Protect shared capacity without treating every caller the same.

Sources: RateLimitFilter; RateLimitPlan; RedisRateLimitStore

A burst is allowed. Sustained overload is not.

Bucket capacity controls bursts; refill controls the long-term rate.



100 burst tokens / 600 refill per minute

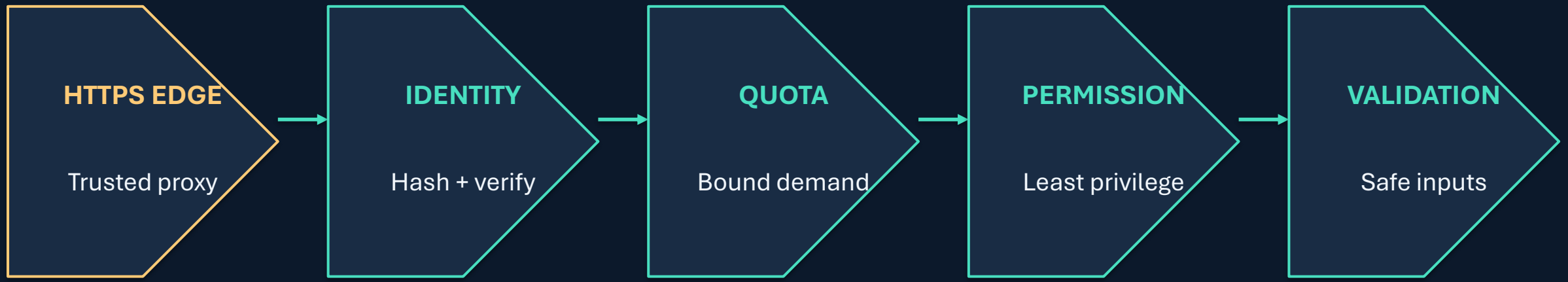
PLAN	REFILL/MIN	BURST
ANONYMOUS	10	10
FREE	60	20
STANDARD	600	100
PARTNER	6,000	500
INTERNAL	6,000	500

Redis fails > local quotas for 30s > retry Redis. Global limits become less strict.

Sources: RateLimitPlan; Buckets; RedisRateLimitStore

Protect the boundary. Minimize privilege.

Application controls and deployment controls work together.



SECRETS OUTSIDE SOURCE

AUDIT ADMIN CHANGES

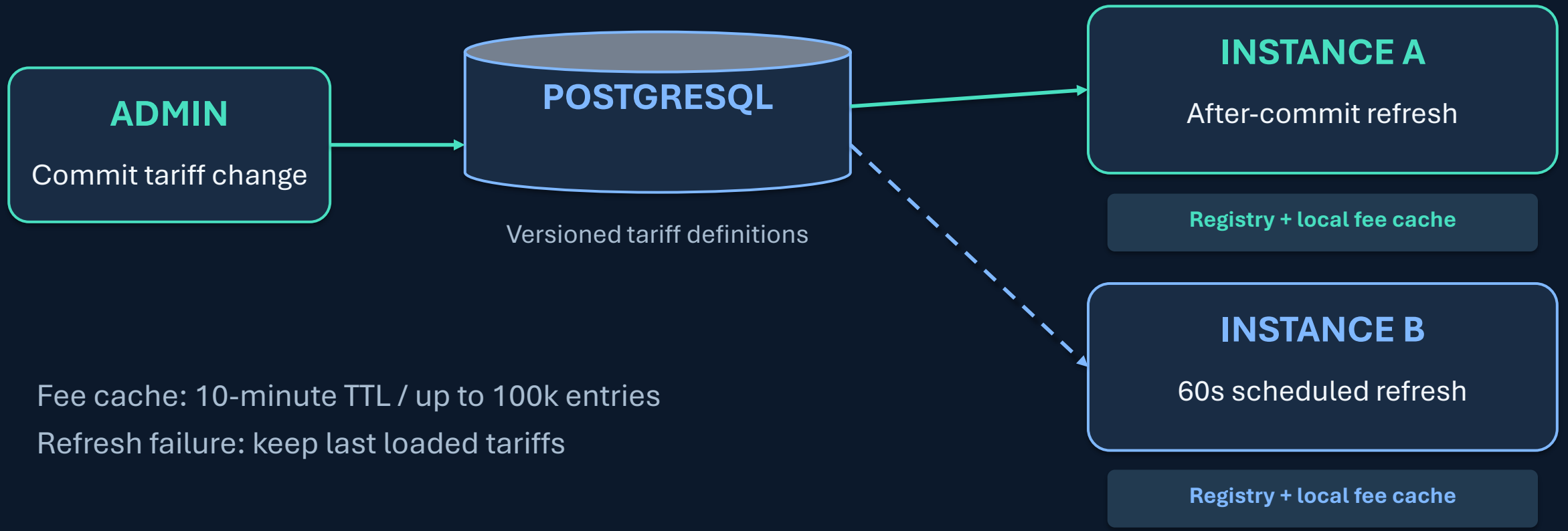
PRIVATE DATA SERVICES

CSRF disabled for header-based stateless auth; revisit if cookies are introduced.

Sources: SecurityConfig; AuthService; ApiKeys; AuditService; README.md

One source of truth. Local speed.

Changes are immediate locally and eventually visible across instances.



Fee cache: 10-minute TTL / up to 100k entries

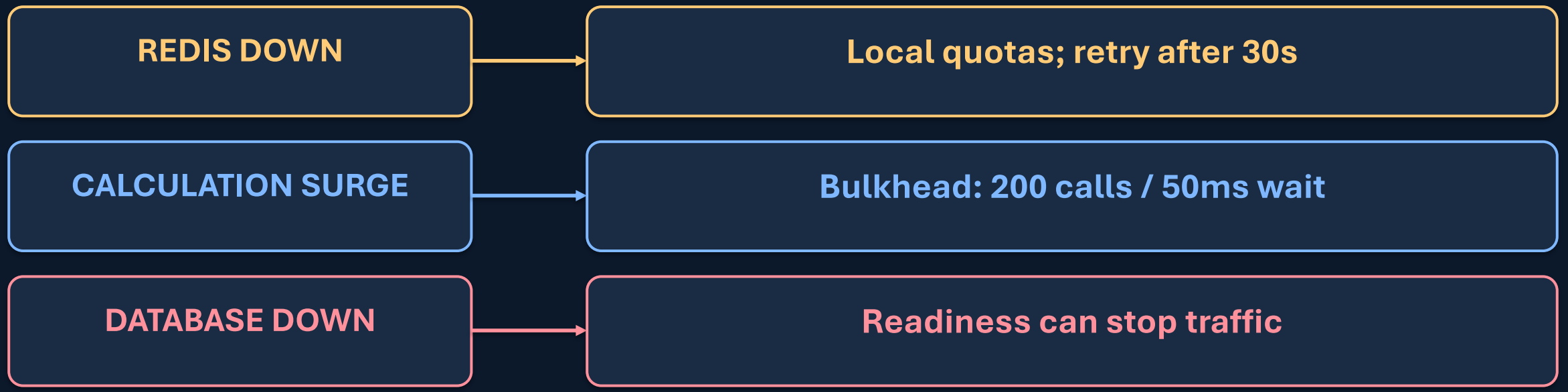
Refresh failure: keep last loaded tariffs

Tariff-version changes clear fee caches when each instance observes the update.

Sources: CachedTariffRegistry; TariffRefreshScheduler; application.yml

Keep serving where it is safe to do so.

Availability is an explicit policy, not a blanket fail-open.



LOGS METRICS TRACES

Liveness: restart? Readiness: receive traffic?

Sources: RedisRateLimitStore; application.yml; TariffsHealthIndicator

Build confidence before shipping code.

Test the rules. Verify integration. Package consistently.



Configured: 90% domain-package coverage gate

Docker absent? Integration tests may skip.

A configured gate is not evidence of a passing build.

Sources: build.gradle; src/test; Dockerfile; README.md

Show the contract in four moments.

Prepared Swagger requests. Local development data only.

01 QUOTE

61 NOK

M1 / 08:00-09:01

02 LOGIN

JWT

Signed access token

03 PERMISSION

403

Missing required authority

04 QUOTA

429

Retry-After header

Quote example: 22 Oct 2026, using the seeded tariff.

Sources: README.md; TariffController; SecurityConfig; RateLimitFilter

A focused foundation. A clear path forward.

Next improvements follow the application's actual tradeoffs.



IDENTITY

External IdP
Token revocation

EDGE PROTECTION

Pre-auth throttling
Redis TLS / cluster support

CONSISTENCY

Cross-instance
cache invalidation

Evolve deliberately. Measure before scaling.

Sources: JwtConfig; ApiClientService; RedisRateLimitStore; README.md

Simple to use. Thoughtful underneath.

Turning complex parking rules into clear, dependable answers.

+

CORRECT

Rules made explicit

+

PROTECTED

Access by design

+

EXTENSIBLE

Built to evolve